CNNP – SKILL 2

THREE-POINT PACKET CAPTURE AND NETWORK PERFORMANCE ANALYSIS

Name: K. Mrudula

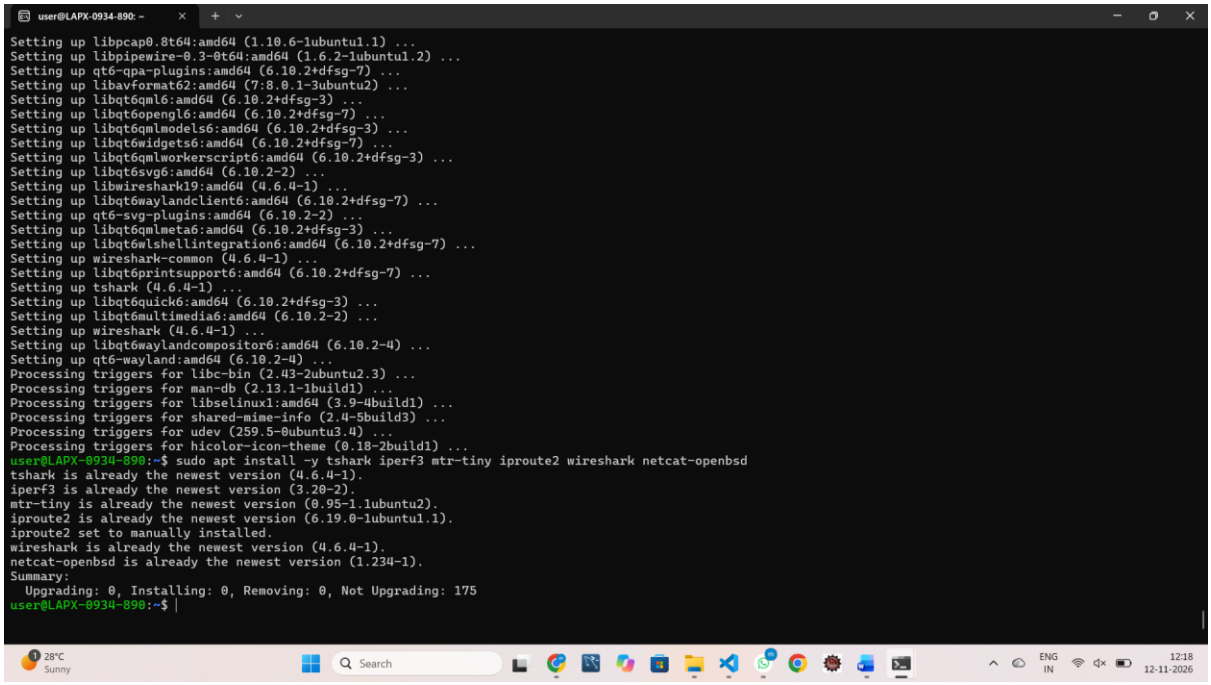
Roll No.: 2520090073

Section: 11

Objective: To create a virtual network using Host1, a router and Host2, capture packets at three points, and analyse packet flow, TTL values, message payload, delay, retransmissions and network performance.

Step 1 – Install the required tools

Updated the Ubuntu package list and installed the required networking and packet-analysis tools: tshark, iperf3, mtr-tiny, iproute2, Wireshark and netcat-openbsd.



Step 2 – Create the network namespaces

Created three isolated network namespaces named host1, router and host2. These namespaces represent the two end systems and the intermediate router.



Step 3 - Create virtual Ethernet links and connect the namespaces

Created virtual Ethernet interfaces to connect Host1 to the router and the router to Host2. A veth pair provides two connected interface endpoints, allowing the namespaces to communicate.

```
root@LAPTOP-1COJKKG6:~# sudo ip link add veth-h1 type veth peer name veth-r1

root@LAPTOP-1COJKKG6:~# sudo ip link add veth-r2 type veth peer name veth-h2
root@LAPTOP-1COJKKG6:~#
```

```
host1
user@LAPX-0934-890:~$ sudo ip link add veth-h1 type veth peer name veth-r1
user@LAPX-0934-890:~$ sudo ip link add veth-r2 type veth peer name veth-h2
user@LAPX-0934-890:~$ sudo ip link set veth-h1 netns host1
user@LAPX-0934-890:~$ sudo ip link set veth-r1 netns router
user@LAPX-0934-890:~$ sudo ip link set veth-r2 netns router
user@LAPX-0934-890:~$ sudo ip link set veth-h2 netns host2
user@LAPX-0934-890:~$
```

Step 4 – Configure Host1 and its interface

Moved the Host1-side interface into the host1 namespace, assigned its IP address and brought the interface up.

```
user@LAPX-0934-890:~$ sudo ip netns exec host2 ip addr add 10.0.2.2/30 dev v
eth-h2
user@LAPX-0934-890:~$ sudo ip netns exec host2 ip link set veth-h2 up
user@LAPX-0934-890:~$ sudo ip netns exec host2 ip link set lo up
user@LAPX-0934-890:~$
```

Step 5 – Enable IP forwarding and routing

Enabled IPv4 forwarding on the router and added routes on Host1 and Host2 so packets for the opposite subnet travel through the router.

```
user@LAPX-0934-890:~$ sudo ip netns exec router sysctl -w net.ipv4.ip_forwar
d=1
net.ipv4.ip_forward = 1
user@LAPX-0934-890:~$ sudo ip netns exec host1 ip route add 10.0.2.0/30 via
10.0.1.2
user@LAPX-0934-890:~$ sudo ip netns exec host2 ip route add 10.0.1.0/30 via
10.0.2.1
user@LAPX-0934-890:~$
```

Step 6 – Test end-to-end connectivity

Sent three ping requests from Host1 to 10.0.2.2. The screenshot shows three replies and 0% packet loss, confirming connectivity between the hosts.

```
user@LAPX-0934-890:~$ sudo ip netns exec host1 ping -c 3 10.0.2.2
PING 10.0.2.2 (10.0.2.2) 56(84) bytes of data.
64 bytes from 10.0.2.2: icmp_seq=1 ttl=63 time=0.603 ms
64 bytes from 10.0.2.2: icmp_seq=2 ttl=63 time=0.070 ms
64 bytes from 10.0.2.2: icmp_seq=3 ttl=63 time=0.051 ms

--- 10.0.2.2 ping statistics ---
3 packets transmitted, 3 received, 0% packet loss, time 2051ms
rtt min/avg/max/mdev = 0.051/0.241/0.603/0.255 ms
user@LAPX-0934-890:~$
```

Step 7 – Add network delay and packet loss

Applied netem traffic control on router interface veth-r2 with a 20 ms delay and 1% packet-loss setting. Displayed the qdisc configuration to verify the applied network conditions.

```
user@LAPX-0934-890:~$ sudo ip netns exec router tc qdisc add dev veth-r2 roo
t netem delay 20ms loss 1%
user@LAPX-0934-890:~$ sudo ip netns exec router tc qdisc show dev veth-r2
qdisc netem 8001: root refcnt 13 limit 1000 delay 20ms loss 1% seed 15271216
450002504350
user@LAPX-0934-890:~$
```

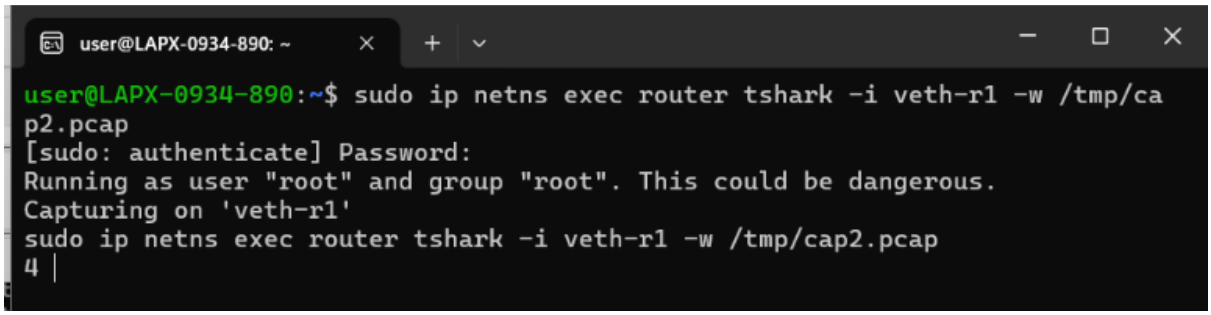
Step 8 – Start packet capture at Host1

Started tshark on veth-h1 and saved the captured packets to /tmp/cap1.pcap. The capture process was left running while test traffic was generated.

```
user@LAPX-0934-890:~$ sudo ip netns exec host1 tshark -i veth-h1 -w /tmp/cap
1.pcap
Running as user "root" and group "root". This could be dangerous.
Capturing on 'veth-h1'
4
```

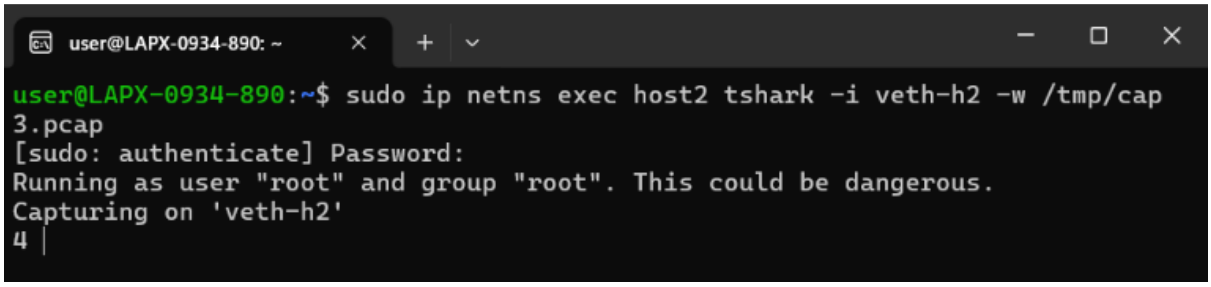
Step 9 - Start packet capture at the router

Started tshark on the router interface veth-r1 and saved packets to /tmp/cap2.pcap.



Step 10 - Start packet capture at Host2

Started tshark on veth-h2 and saved packets to /tmp/cap3.pcap. Capturing at all three points allows the same traffic to be compared along the network path.



Step 11 – Send a TCP test message from Host1 to Host2

Connected from Host1 to Host2 using netcat on TCP port 1234 and sent the test messages shown in the screenshot: “Hello host!”, “This is my lab experiment”, “Computer Networks Lab” and “Three Point Capture Experiment”.

```
user@LAPX-0934-890:~$  sudo ip netns exec host1 nc 10.0.2.2 1234
Hello host!
This is my lab experiment
Computer Networks Lab
Three Point Capture Experiment
user@LAPX-0934-890:~$
```

```
user@LAPX-0934-890: ~
user@LAPX-0934-890:~$ sudo ip netns exec host2 nc -l 1234
[sudo: authenticate] Password:
Hello host!
Computer Networks Lab
Three Point Capture Experiment
```

Step 12 – Check and preserve the capture files

Changed the permissions of the three pcap files so they could be read and inspected, then listed their sizes to confirm that the files existed.

```
user@LAPX-0934-890: ~
user@LAPX-0934-890:~$ sudo chmod 644 /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3
.pcap
[sudo: authenticate] Password:
user@LAPX-0934-890:~$ ls -lh /tmp/cap1.pcap /tmp/cap2.pcap /tmp/cap3.pcap
-rw-r--r-- 1 root root 6.2K Nov 12 09:34 /tmp/cap1.pcap
-rw-r--r-- 1 root root 5.7K Nov 12 09:34 /tmp/cap2.pcap
-rw-r--r-- 1 root root 5.7K Nov 12 09:34 /tmp/cap3.pcap
user@LAPX-0934-890:~$
```

Step 13 - Analyse IPv4 TTL values in the three captures

Used tshark to extract and count IPv4 TTL values from cap1.pcap, cap2.pcap and cap3.pcap. The displayed results include TTL values 63 and 64 at the capture points.

```
user@LAPX-0934-890:~$ tshark -r /tmp/cap1.pcap -T fields -e ip.ttl -Y "ip" 2
>/dev/null | sort | uniq -c
     16 63
     18 64
user@LAPX-0934-890:~$ tshark -r /tmp/cap2.pcap -T fields -e ip.ttl -Y "ip" 2
>/dev/null | sort | uniq -c
     16 63
     18 64
user@LAPX-0934-890:~$ tshark -r /tmp/cap3.pcap -T fields -e ip.ttl -Y "ip" 2
>/dev/null | sort | uniq -c
     18 63
     16 64
user@LAPX-0934-890:~$
```

Step 14 - Read the TCP payload from the router capture

Filtered the router capture for TCP port 1234 and extracted the packet payload. The hexadecimal output was converted for inspection, showing the text sent in the TCP test message.

```
user@LAPX-0934-890:~$ tshark -r /tmp/cap2.pcap -Y "tcp.port == 1234" -T fiel
ds -e tcp.payload 2>/dev/null | xxd | head -20
00000000: 0a0a 0a34 3836 3536 6336 6336 6632 3036  ...48656c6c6f206
00000010: 3836 6637 3337 3432 3130 610a 0a35 3436  86f7374210a..546
00000020: 3836 3937 3332 3036 3937 3332 3036 6437  86973206973206d7
00000030: 3932 3036 6336 3136 3232 3036 3537 3837  9206c61622065787
00000040: 3036 3537 3236 3936 6436 3536 6537 3430  06572696d656e740
00000050: 6134 3336 6636 6437 3037 3537 3436 3537  a436f6d707574657
00000060: 3232 3034 6536 3537 3437 3736 6637 3236  2204e6574776f726
00000070: 6237 3332 3034 6336 3136 3230 610a 0a0a  b73204c61620a...
00000080: 0a0a 3438 3635 3663 3663 3666 3230 3638  ..48656c6c6f2068
00000090: 3666 3733 3734 3231 3061 0a0a 3534 3638  6f7374210a..5468
000000a0: 3639 3733 3230 3639 3733 3230 3664 3739  6973206973206d79
000000b0: 3230 3663 3631 3632 3230 3635 3738 3730  206c616220657870
000000c0: 3635 3732 3639 3664 3635 3665 3734 3061  6572696d656e740a
000000d0: 0a0a 3433 3666 3664 3730 3735 3734 3635  ..436f6d70757465
000000e0: 3732 3230 3465 3635 3734 3737 3666 3732  72204e6574776f72
000000f0: 3662 3733 3230 3463 3631 3632 3061 0a0a  6b73204c61620a..
00000100: 3534 3638 3732 3635 3635 3230 3530 3666  546872656520506f
00000110: 3639 3665 3734 3230 3433 3631 3730 3734  696e742043617074
00000120: 3735 3732 3635 3230 3435 3738 3730 3635  7572652045787065
00000130: 3732 3639 3664 3635 3665 3734 3061 0a0a  72696d656e740a..
user@LAPX-0934-890:~$
```

Step 15 - Compare packet timestamps at Host1 and Host2

Extracted frame.time_epoch timestamps for TCP data packets from cap1.pcap and cap3.pcap. The displayed timestamps can be compared to observe the time difference as packets travel across the network.

```
user@LAPX-0934-890:~$ tshark -r /tmp/cap1.pcap -Y "tcp.len > 0" -T fields -e
 frame.time_epoch 2>/dev/null | head
1794475699.052794493
1794475699.074783935
1794475748.883279593
1794475755.712100406
1794475761.308555920
1794475767.292721757
1794476010.684501397
1794476017.746901184
1794476026.244984661
user@LAPX-0934-890:~$ tshark -r /tmp/cap3.pcap -Y "tcp.len > 0" -T fields -e
 frame.time_epoch 2>/dev/null | head
1794475699.074532134
1794475699.094874258
1794475748.903761979
1794475755.732381294
1794475761.328675594
1794475767.313070535
1794476010.704804120
1794476017.767346033
1794476026.265264814
user@LAPX-0934-890:~$
```

Step 16 - Check TCP retransmission indications

Ran the tshark filter tcp.analysis.retransmission on the Host1 capture to check for TCP retransmission indications.

```
user@LAPX-0934-890:~$ tshark -r /tmp/cap1.pcap -Y "tcp.analysis.retransmissi
on"
```

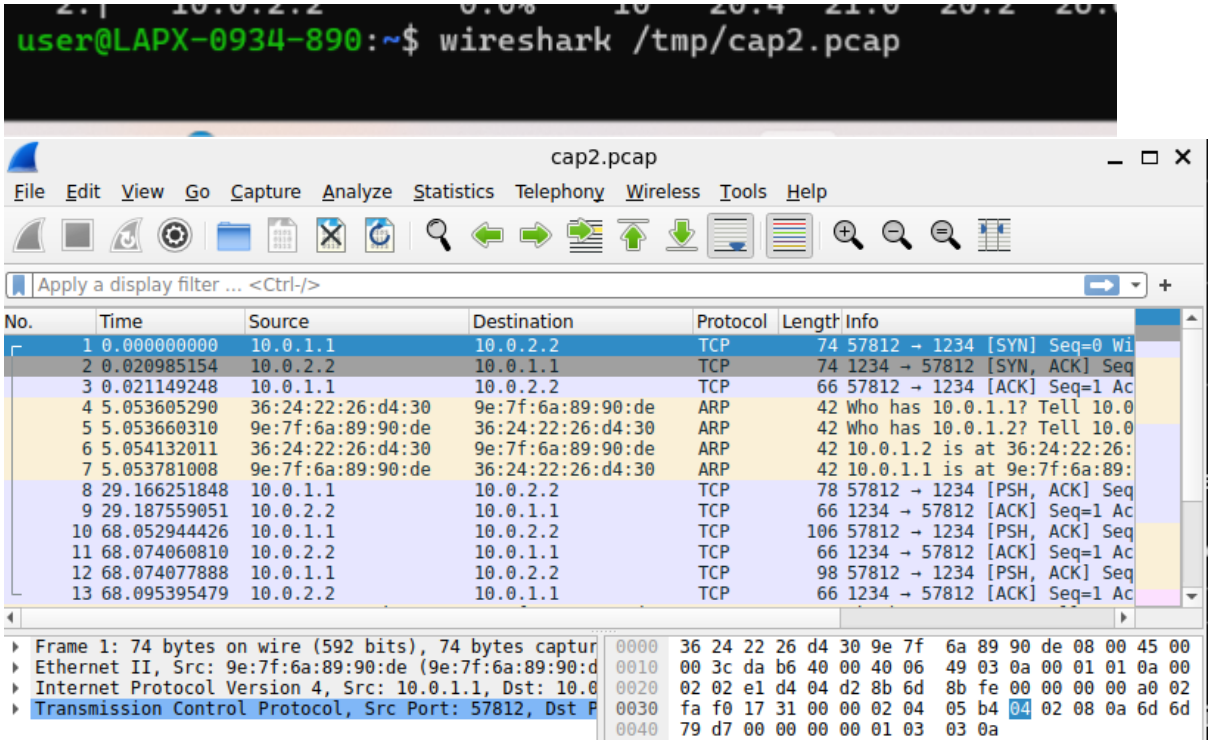
Step 17 - Measure RTT and packet loss using MTR

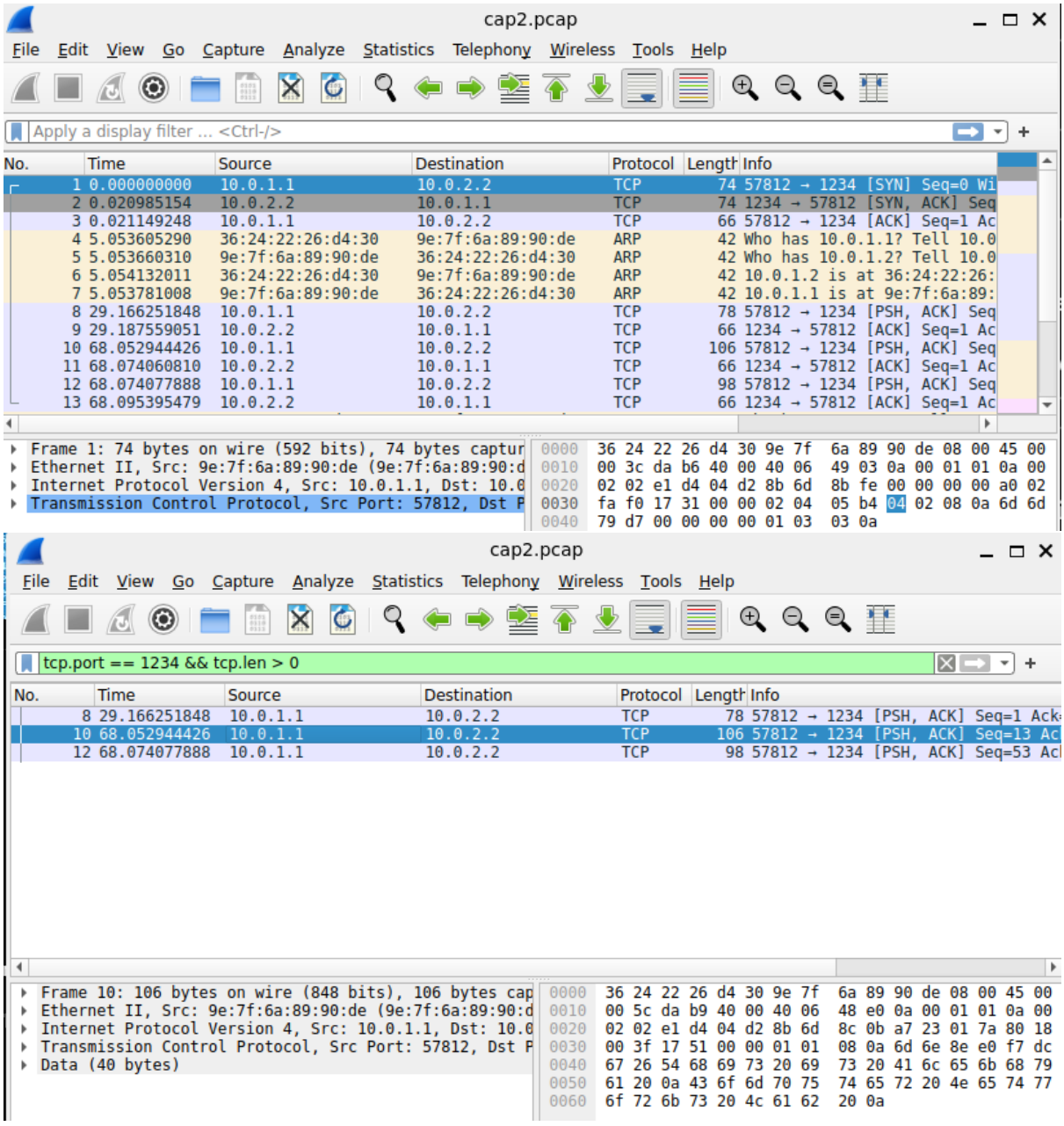
Ran MTR from the host1 namespace to 10.0.2.2 with 10 probes. The screenshot shows 0.0% loss at the router hop and 0.0% loss at Host2 for this run, with an average RTT of about 21.0 ms at Host2.

```
user@LAPX-0934-890:~$ sudo ip netns exec host1 mtr -rw -c 10 10.0.2.2
Start: 2026-11-12T09:40:34+0000
HOST: LAPX-0934-890 Loss%   Snt   Last   Avg  Best  Wrst StDev
  1.|-- 10.0.1.2       0.0%    10    0.1   0.2   0.0   0.8   0.2
  2.|-- 10.0.2.2       0.0%    10   20.4  21.0  20.2  26.6   2.0
user@LAPX-0934-890:~$
```

Step 18 - Open the router packet capture in Wireshark

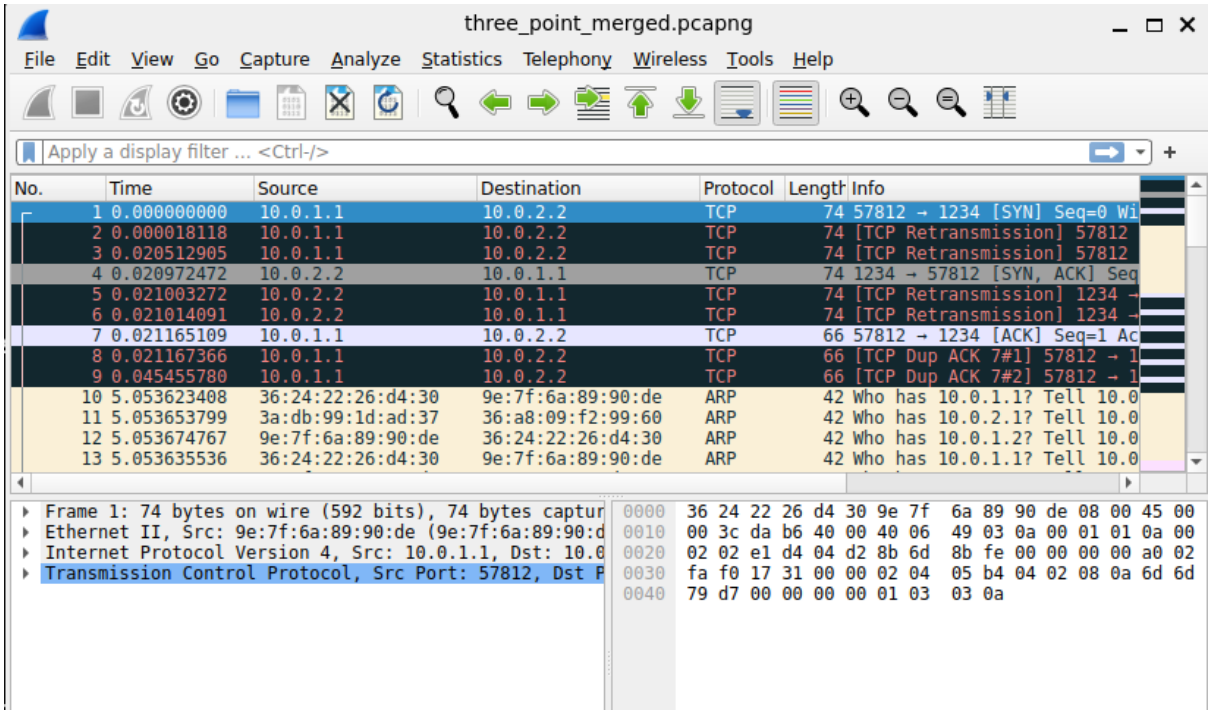
Opened /tmp/cap2.pcap in Wireshark to inspect the captured packets and their protocol details.





Step 19 – Merge the three packet captures

Used mergecap to combine cap1.pcap, cap2.pcap and cap3.pcap into /tmp/three_point_merged.pcapng. capinfos verified 53 packets across three interfaces: 18 packets on veth-h1, 18 on veth-r1 and 17 on veth-h2.



Step 20 – Review Wireshark Expert Information

Opened Analyze → Expert Information for the merged capture. Wireshark displayed suspected fast retransmission, duplicate ACK and suspected retransmission notes. Because the file combines observations from multiple capture interfaces, repeated observations of the same packet can produce analysis indications; these notes should not be treated alone as proof of end-to-end retransmission.

